

Git & GitHub 实战笔记

从推送失败到强制替换远程仓库的完整指南

学习笔记

2026 年 3 月 21 日

目录

1	引言	3
2	起点：本地文件准备	3
3	关联远程仓库	3
3.1	比喻：两种连接方式	3
4	第一次障碍：SSH 公钥认证失败	3
4.1	原因分析	4
4.2	解决步骤	4
4.2.1	1. 检查已有 SSH 密钥	4
4.2.2	2. 生成新密钥（如无）	4
4.2.3	3. 将私钥加入 SSH 代理	4
4.2.4	4. 将公钥添加到 GitHub	4
4.2.5	5. 测试连接	4
4.3	比喻：配钥匙	4
5	第二次障碍：远程仓库已有内容	5
5.1	解决方案：拉取并合并	5
5.1.1	参数解释	5
6	新问题：分支名不匹配	5
6.1	解决方法	5
6.1.1	方法一：重命名本地分支	5
6.1.2	方法二：直接推送本地 master 到远程 main	5
6.2	比喻：分岔路口	6

7 最终场景：用本地新仓库完全替换远程仓库	6
7.1 注意事项	6
7.2 比喻：换房搬家	7
8 清理无用文件	7
8.1 修正提交	7
8.2 比喻：打包行李	7
9 最终验证	7
10 常见问题与解决	7
11 总结	8

1 引言

在使用 Git 和 GitHub 的过程中，我们经常需要将本地项目推送到远程仓库。但新手往往会遇到 SSH 认证失败、远程仓库已有内容、分支名不匹配等一系列问题。本文将用一个真实案例，详细记录从零开始创建本地仓库、解决 SSH 密钥问题、处理远程冲突，直到最终用本地新仓库完全替换远程内容的完整过程。通过生动形象的比喻，帮助理解每个步骤背后的原理。

2 起点：本地文件准备

假设我们在 Documents 文件夹下有一些笔记文件（如 PDF），希望将它们上传到 GitHub 上进行备份。

```
1 cd ~/Documents
2 # 假设已有文件：fedora-proxy-configuration-note.pdf 等
3 git init
4 git add .
5 git commit -m "upload"
```

此时本地 Git 仓库已经建立，提交历史中包含这些文件。

3 关联远程仓库

使用 SSH 方式关联 GitHub 仓库：

```
1 git remote add origin git@github.com:用户名/仓库名.git
```

3.1 比喻：两种连接方式

- SSH：好比你有自己家的钥匙，每次回家不用按门铃，但需要先配好钥匙。
- HTTPS：好比用密码开门，每次需要输入账号密码（或令牌）。

4 第一次障碍：SSH 公钥认证失败

执行推送时出现：

```
1 git@github.com: Permission denied (publickey).
2 fatal: Could not read from remote repository.
```

4.1 原因分析

GitHub 不认识你的电脑，拒绝连接。就像你去一个小区，保安没有你的指纹记录，不让你进门。

4.2 解决步骤

4.2.1 1. 检查已有 SSH 密钥

```
1 ls -al ~/.ssh
```

如果看到 `id_ed25519` 和 `id_ed25519.pub` 等文件，说明已有密钥；否则需要生成。

4.2.2 2. 生成新密钥（如无）

```
1 ssh-keygen -t ed25519 -C "your-email@example.com"
```

一路回车即可（可以设置密码短语，增加安全性）。

4.2.3 3. 将私钥加入 SSH 代理

```
1 eval "$(ssh-agent -s)"
2 ssh-add ~/.ssh/id_ed25519
```

4.2.4 4. 将公钥添加到 GitHub

```
1 cat ~/.ssh/id_ed25519.pub
```

复制输出的全部内容，登录 GitHub → Settings → SSH and GPG keys → New SSH key，粘贴并保存。

4.2.5 5. 测试连接

```
1 ssh -T git@github.com
```

成功时会显示：Hi 用户名! You've successfully authenticated...

4.3 比喻：配钥匙

生成密钥对就像配了一把钥匙（私钥），并把钥匙的齿纹（公钥）交给管理员。以后每次用私钥开门，管理员都会验证你的身份。

5 第二次障碍：远程仓库已有内容

SSH 认证通过后，再次推送时遇到：

```
1 ! [rejected]          main -> main (fetch first)
2 error: failed to push some refs to ...
```

这是因为远程仓库已经存在一些文件（如自动生成的 README.md、LICENSE），而本地仓库没有这些内容。Git 为了保护远程数据，拒绝直接覆盖。

5.1 解决方案：拉取并合并

```
1 git pull origin main --allow-unrelated-histories --no-rebase
```

5.1.1 参数解释

- `--allow-unrelated-histories`：允许合并没有共同祖先的分支。本地是新仓库，远程有独立历史，需要此参数。
- `--no-rebase`：使用合并（merge）而非变基（rebase），保留双方历史。

合并后，远程的文件和本地的文件会同时存在。然后就可以正常推送：

```
1 git push -u origin main
```

6 新问题：分支名不匹配

在某些情况下，本地默认分支是 master，而远程是 main。推送时指定 main 会报错：

```
1 error: src refspec main does not match any
```

6.1 解决方法

6.1.1 方法一：重命名本地分支

```
1 git branch -m master main
2 git push -u origin main
```

6.1.2 方法二：直接推送本地 master 到远程 main

```
1 git push -u origin master:main
```

6.2 比喻：分岔路口

分支名就像不同的路线。你本来在“master”路上，却告诉别人要去“main”路的终点，导航当然会迷路。你需要先把自己的路名改对，或者明确告诉它“master”对应“main”。

7 最终场景：用本地新仓库完全替换远程仓库

如果你发现远程仓库的内容已经过时或混乱，想用本地的新仓库彻底覆盖远程，可以使用强制推送。

```
1 # 进入本地仓库目录
2 cd ~/Documents/My-Knowledge-Base-Master
3
4 # 如果该目录之前是某个 Git 仓库的子目录，可以删除原有的 .git 重新初始化
5 rm -rf .git # 小心：这会清除本地 Git 历史
6
7 # 重新初始化并提交
8 git init
9 git add .
10 git commit -m "Move to a new repository"
11
12 # 关联远程仓库
13 git remote add origin git@github.com:maze01001000/My-Knowledge-Base.git
14
15 # 强制推送，覆盖远程所有内容
16 git push -u origin main --force
```

7.1 注意事项

- 强制推送会彻底删除远程仓库的所有历史记录和文件，仅保留最后一次推送的内容。
- 只有在你确认远程内容不再需要时才使用此操作。
- 如果其他协作者已克隆该仓库，他们需要重新克隆，因为历史已被重写。

7.2 比喻：换房搬家

正常推送就像在旧房子上加盖楼层，保留原有地基。强制推送则像推倒旧房子，在原址上建一座全新的房子。旧房子里的一切都将不复存在。

8 清理无用文件

在提交时，有时会误将编辑器备份文件（如`~`）或 SSH 密钥副本（如`~.pub`）加入暂存区。这些文件不应纳入版本控制。

8.1 修正提交

```
1 # 从暂存区移除无用文件
2 git rm --cached ~ ~.pub
3
4 # 物理删除（可选，如果文件存在）
5 rm ~ ~.pub
6
7 # 修正上一次提交（将移除操作合并到之前的提交中）
8 git commit --amend -m "Move to a new repository"
```

8.2 比喻：打包行李

提交就像打包行李。如果不小心把垃圾也塞进箱子，就需要打开箱子清理掉，再重新封箱。

9 最终验证

推送成功后，在浏览器中打开远程仓库页面，应该能看到本地所有文件。如果仍有问题，可以：

- 检查远程仓库 URL 是否正确：`git remote -v`
- 确认默认分支设置（GitHub 仓库 Settings → Branches）
- 刷新页面，有时浏览器缓存会显示旧内容

10 常见问题与解决

Q1: 问：为什么 `git pull` 时需要 `--allow-unrelated-histories`?

Q2: 答: 因为本地和远程是两个没有共同祖先的独立仓库。Git 默认拒绝合并这样的分支, 加上该选项表示 “我知道它们无关, 但请合并”。

Q3: 问: 强制推送后, 还能找回远程之前的内容吗?

Q4: 答: 如果没有其他备份, 通常无法找回。Git 的垃圾回收机制会在一段时间后清除被覆盖的对象, 因此强制推送前务必三思。

Q5: 问: 我本地是 master, 远程是 main, 如何让它们保持一致?

Q6: 答: 推荐将本地分支重命名为 main, 这样后续操作更清晰。也可以在 GitHub 仓库设置中将默认分支改为 master。

Q7: 问: 提交时忘记写 -m 消息, 导致提交中止, 怎么办?

Q8: 答: 重新执行 `git commit -m " 消息 "` 即可。如果已经执行了 `git commit` 而没有消息, 编辑器会打开, 保存并退出即可完成提交。

11 总结

通过这次实战, 我们学习了:

- 如何配置 SSH 密钥并关联 GitHub。
- 如何处理远程仓库已存在内容的冲突 (pull 合并)。
- 分支名不匹配时的解决方法 (重命名或显式映射)。
- 如何使用强制推送完全替换远程仓库内容。
- 如何修正提交中的错误文件 (`git rm --cached` 和 `git commit --amend`)。

Git 的设计理念是保护数据安全, 但也提供了强大的工具来应对各种场景。掌握这些基础操作后, 你就能自信地管理自己的代码仓库。

版本控制, 从此不再迷路!